

Analyse des registres de configuration d'un périphérique, le GPIO

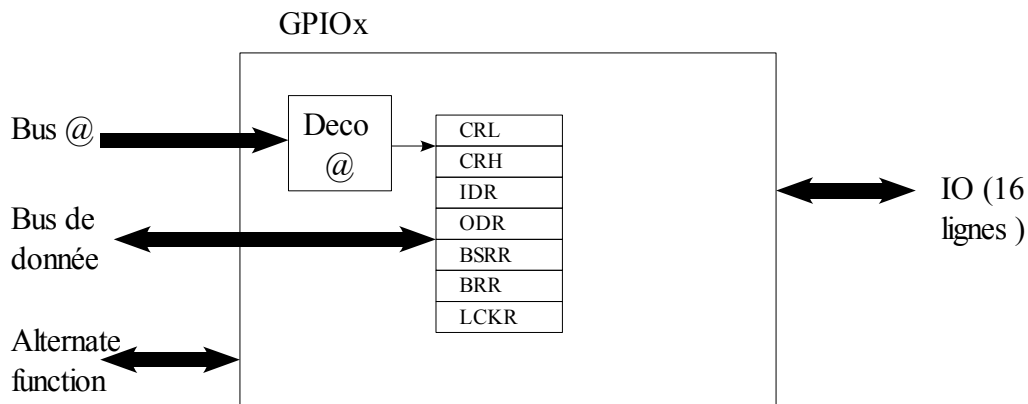
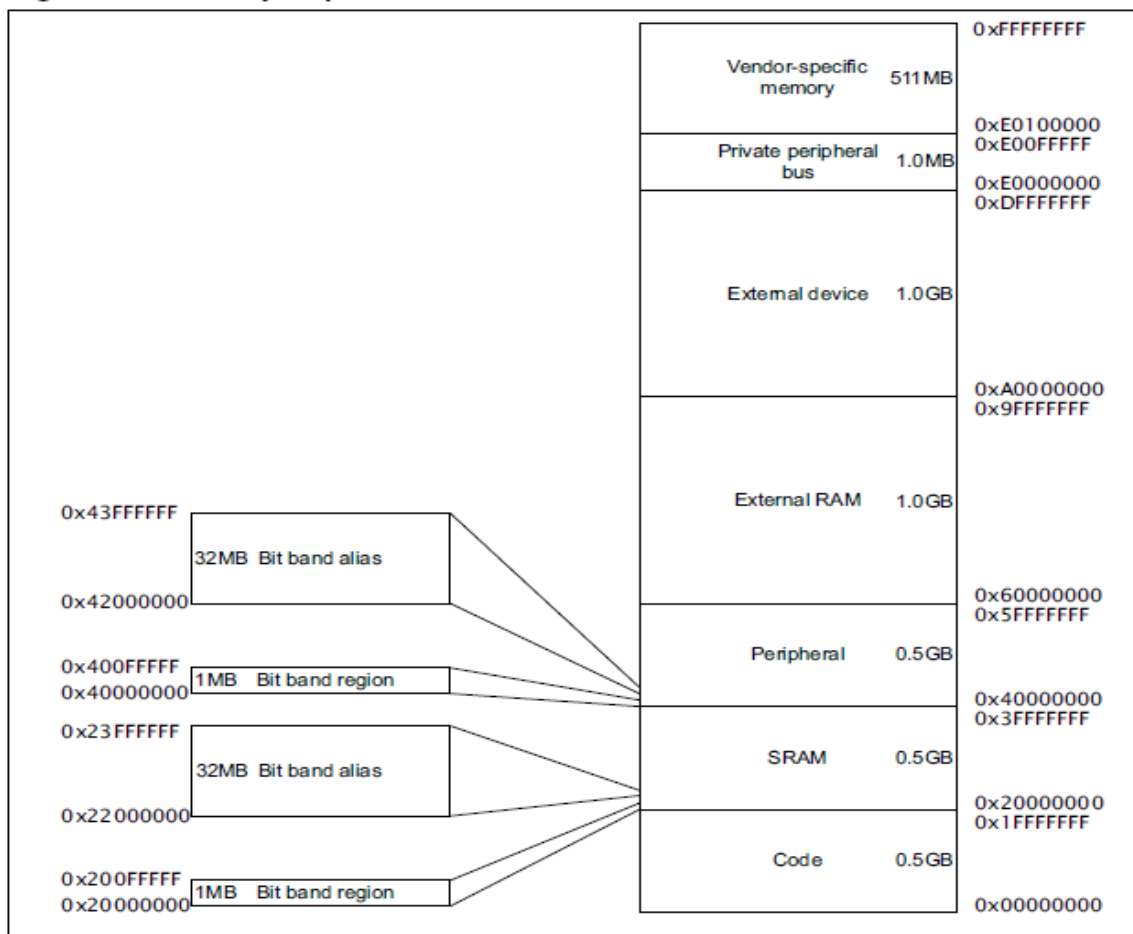


Figure 1 : Schéma simplifié du GPIO de ST, qui est connecté au Cortex M3

Rappelons, ci-dessous le mapping mémoire du Cortex M3 :

Figure 9. Memory map



On note que l'espace réservé aux périphériques, de type RAM, se situe à partir de l'adresse 0x4000 0000.

La philosophie adoptée est de considérer pour le GPIO, non pas 7 registres, mais un set de registres. Ainsi, une adresse unique est affectée au périphérique, celle du premier registre du jeu de registres, ici, *CRL*. Les autres sont systématiquement placés en suivant dans le plan mémoire.

Donc de manière naturelle, on associe à chaque périphérique une **structure de registres**. Pour le GPIO, cette structure est définie par (extrait de *stmregs.h*) :

```
typedef struct
{
    vu32 CRL;
    vu32 CRH;
    vu32 IDR;
    vu32 ODR;
    vu32 BSRR;
    vu32 BRR;
    vu32 LCKR;
} GPIO_TypeDef;
```

Ce morceau de code est la création d'un nouveau **type**, une structure, dont le nom (du type) est **GPIO_TypeDef**. C'est un type au même titre que *int* par exemple.

GPIO_TypeDef peut être vu comme un tableau à 7 cases de type **vu32** (*volatile unsigned long int*), un entier non signé 32 bits en RAM pour faire simple !

Supposons maintenant que nous ayons à faire au GPIO A. Supposons aussi que le fondeur de la puce, ST, ait choisi comme adresse de base, 0x4000 0800 que nous appellerons pour plus de clarté, *GPIOA_BASE*. La question qui se pose est : comment avoir accès aux 7 registres.

Première réponse :

Il faudrait arriver à créer une variable *GPIOA*, de type *GPIO_TypeDef* et surtout, il faut mapper cette variable à l'adresse 0x4000 0800. En clair *GPIOA* doit avoir l'adresse de son premier élément en *GPIOA_BASE*. C'est faisable, mais pas simple : il faudrait contraindre l'éditeur de lien en l'obligeant à affecter *GPIOA* à l'adresse *GPIOA_BASE*.

Seconde réponse :

Créer un pointeur (comme lors d'un accès à un tableau) dont le nom est *GPIOA* et qui donc pointe sur un type *GPIO_TypeDef* :

```
GPIO_TypeDef *GPIOA;
GPIOA = GPIOA_BASE;
```

La dernière ligne génère une erreur. En effet, *GPIOA* ne peut recevoir que l'adresse d'un objet de type *GPIO_TypeDef*. Pour parvenir à faire cette affectation, il faut faire un *casting*. C'est à dire qu'il faut explicitement préciser que *GPIOA_BASE* est l'adresse d'une variable de type *GPIO_TypeDef*. On obtient donc :

```
GPIO_TypeDef *GPIOA;
GPIOA = (GPIO_TypeDef*) GPIOA_BASE;
```

(GPIO_TypeDef*) GPIOA_BASE peut être vu comme une valeur immédiate (dans notre cas l'adresse 0x4000 0800), interprétée par le compilateur comme un pointeur sur la variable de type *GPIO_TypeDef* située à l'adresse *GPIOA_BASE*.

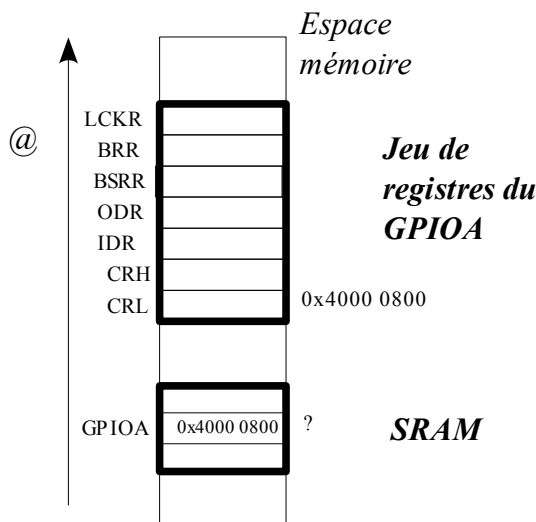
Ensuite, il faut pouvoir utiliser ce pointeur. Cela s'obtient par :

```
(*GPIOA).ODR=0x0011;
```

Mais, de manière plus rapide est plus lisible on peut écrire :

```
GPIOA->ODR=0x0011;
```

De manière schématique :



Cette solution est tout à fait satisfaisante, sauf qu'elle oblige à utiliser une case mémoire (*long int*) pour le pointeur *GPIOA*. On peut arriver au même résultat sans utiliser de case RAM.

Troisième réponse :

Pour économiser une case mémoire, on peut exploiter directement le pointeur ((GPIO_TypeDef *) GPIOA_BASE). Par exemple :

```
((GPIO_TypeDef *) GPIOA_BASE) → ODR = 0x0002;
```

Mais c'est pas beau ! Donc on crée une équivalence :

```
#define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
```

ce qui nous permet d'écrire :

```
GPIOA->ODR = 0x20;
```

Par la suite, *GPIOA* sera donc vu comme un pointeur sur la structure de registre du port A, mais sans existence physique (pas de case RAM allouée).